

Training Leaves Traces: Centered Residual Signatures for Language Model Lineage Verification

Anonymous ACL submission

Abstract

Open-weight language models are fine-tuned, quantized, pruned, and merged, yet their provenance is often undocumented. We study data-free white-box lineage verification: **can weights alone reveal whether two compatible residual checkpoints share weight-level ancestry?**

Residual training produces a shared identity-aligned component in branch products, so this structure alone cannot establish ancestry. We remove it and compare checkpoint-specific structure across residual blocks, yielding a symmetric lineage score calibrated against independent checkpoints. On residual-MLP and GPT-2 benchmarks, the score separates fine-tuned, LoRA-merged, pruned, and quantized descendants from independent and distilled models (AUROC=1.0), distinguishing weight ancestry from behavioral similarity. Under function-preserving checkpoint laundering, weight-space baselines lose margin or fail; our score remains unchanged and runs $76\times$ faster than the nearest robust baseline on GPT-2. The projection-pairing signal appears across six language-model families, and a LLaMA-2 case study matches documented relationships. Taken together, these results establish a passive, data-free provenance signal for compatible open-weight language-model checkpoints.

1 Introduction

Open-weight language models are increasingly distributed through supply chains rather than as isolated releases. A base checkpoint may be fine-tuned (Devlin et al., 2019; Ouyang et al., 2022), quantized (Han et al., 2016; Dettmers et al., 2022), pruned (Han et al., 2015), adapted with low-rank updates, merged (Wortsman et al., 2022; Ilharco et al., 2023), and redistributed under a new name. These operations create weight descendants, but their ancestry may be undocumented or intentionally obscured. This makes it difficult to distinguish

a true descendant from a model that merely behaves similarly.

We study post-hoc white-box model lineage verification. Given reference and suspect checkpoints with compatible residual architectures, the verifier determines from weights alone whether they share weight-level ancestry. Checkpoints are related when they inherit from a common weight ancestor through operations such as fine-tuning, RLHF, pruning, quantization, or LoRA merging. Independently initialized checkpoints are unrelated, even when they share architecture, data, task, or outputs; a distilled student trained from fresh weights is therefore not a weight descendant of its teacher. The task is symmetric: it establishes shared ancestry, not direction of descent.

Existing mechanisms do not solve this setting. Hashes fail after any weight change. Metadata, model cards (Mitchell et al., 2019), and proof-of-learning records (Jia et al., 2021) require trustworthy documentation. Watermarks (Uchida et al., 2017; Adi et al., 2018) must be inserted proactively. Behavioral and representation-based methods such as IPGuard (Cao et al., 2021), CKA (Kornblith et al., 2019), and SVCCA (Raghu et al., 2017) require data and forward passes and measure similarity rather than weight inheritance. Direct weight similarity is data-free but can be disrupted by function-preserving reparameterizations.

Our starting point is a structural property of trained residual networks. Park (2026) observed that correctly paired input and output projections form branch products with concentrated trace. Yet trace concentration alone cannot establish ancestry because independently trained residual models acquire the same generic identity-aligned component. We decompose each branch product into this shared component and an orthogonal, traceless remainder. The normalized remainder forms a checkpoint-specific centered residual signature. Aligning signatures across blocks and averaging

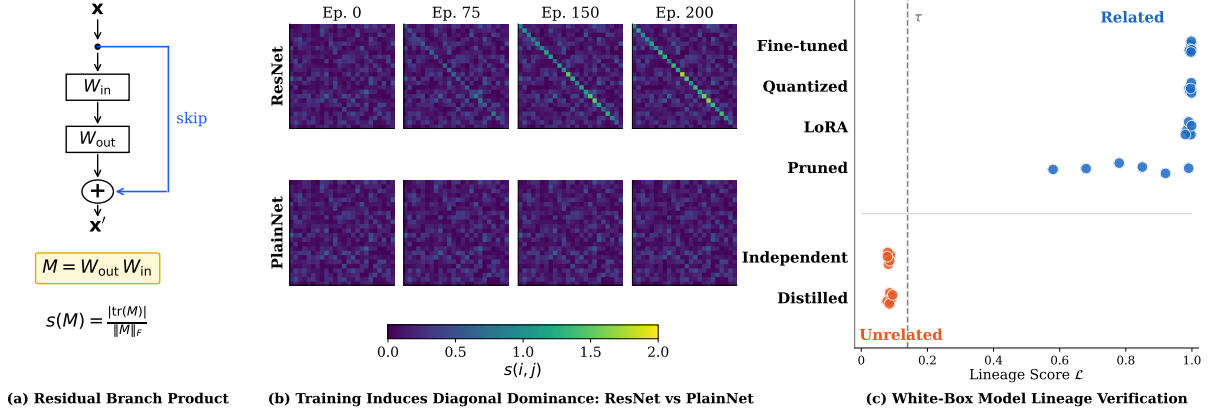


Figure 1: Centered residual signatures for lineage verification. (a) The residual branch product $M = W_{\text{out}}W_{\text{in}}$ for a block with skip connection; the normalized trace concentration $s(M)$ measures identity alignment. (b) Training dynamics: score matrices $s(i, j)$ from epoch 0 to 200. ResNet (top row) develops strong identity-aligned structure; PlainNet without skip connections (bottom row) remains random. The structure is learned, not an initialization artifact. (c) Lineage verification: the lineage score $\mathcal{L}$ (Eq. 6) aggregates per-block centered-signature similarity. Descendants (fine-tuned, quantized, pruned, LoRA-merged) score high; non-descendants (independent, distilled) score near zero. Note: simple weight-space baselines also solve unlaundered cases; the invariance property distinguishes the proposed method (Section 4.2).

their similarities yields a symmetric lineage score with empirical-null calibration.

The composed branch product removes important coordinate symmetries. Hidden-unit permutations and reciprocal rescaling can alter individual matrices while preserving the block function, but cancel in the product. The resulting signature is algebraically invariant without alignment search. Controlled interventions further show that the trace-concentrated structure is learned, requires skip connections, and is consistent with accumulated within-block gradient coupling.

Across six language model families, architecture-aware products recover projection pairings with 80–100% accuracy (the phenomenon also appears in vision and speech architectures; Appendix E). On controlled residual-MLP and approximately 35M-parameter GPT-2 benchmarks, centered residual signatures separate fine-tuned, LoRA-merged, pruned, and quantized descendants from independently trained and distilled models with AU-ROC=1.0. Simple weight-space baselines also solve clean cases. Under function-preserving checkpoint laundering, however, raw-coordinate methods lose margin or fail, while our score remains invariant. Re-Basin with scale recovery remains accurate but requires cubic-time alignment; our method matches its discrimination at $76\times$ lower latency on GPT-2. A public LLaMA-2 case study comparing 3 documented descen-

dants against 7 independently trained architectural clones—OpenLLaMA, Amber, Baichuan, InternLM, and Yi—shows a $\sim 10,000\times$ score gap, providing strong evidence that the method distinguishes true weight ancestry from mere architectural similarity.

Contributions.

1. We formulate data-free pairwise weight-lineage verification and introduce centered residual signatures that isolate checkpoint-specific structure.
2. We construct a symmetric model-level score with empirical-null calibration and algebraic invariance to hidden-unit permutation and reciprocal rescaling.
3. We evaluate robustness to post-training transformations, checkpoint laundering, and distillation, and demonstrate projection-pair recovery across six language model families.

2 Problem Setting and Related Work

2.1 White-Box Model Lineage Verification

A verifier receives two checkpoints A and B and determines whether they share weight-level lineage. We assume white-box access and compatible residual architectures (same L and d); cross-architecture comparisons are out of scope. The verifier operates on weights alone: no training data, activations, or

Method	Key Limitation
Crypto Hash	Breaks after any weight change
Metadata/Logs	Requires honest distributor
Watermarking	Must insert before release
Behavioral (CKA, etc.)	Needs data and forward passes
Weight Similarity	Breaks under reparameterization
Re-Basin	$O(d^3)$ alignment per pair
Ours	<i>Residual architectures only</i>

Table 1: Each prior method has a limitation for retroactive, data-free lineage verification. Our method addresses these but is restricted to residual architectures.

forward passes.

Checkpoints are *related* if they share a common weight ancestor through fine-tuning, RLHF, pruning, quantization, or LoRA merging. They are *unrelated* if initialized independently, even when architecture, data, task, or outputs match; a distilled student trained from scratch is unrelated to its teacher, even if it mimics the outputs perfectly. Linear merges induce partial lineage (Appendix D.2). The verifier returns RELATED or UNRELATED. The score is symmetric, $\mathcal{L}(A, B) = \mathcal{L}(B, A)$; directional attribution requires metadata (Appendix B). The method complements cryptographic signing and watermarking where those were never applied.

2.2 Related Work

Table 1 summarizes the comparison.

Cryptographic hashes break after any weight modification. Model cards (Mitchell et al., 2019) require honest distributors and can be falsified when checkpoints are redistributed. Proof-of-learning (Jia et al., 2021) requires retained training transcripts that already-released checkpoints lack.

Parameter watermarks (Uchida et al., 2017) and backdoor watermarks (Adi et al., 2018) embed deliberate signals during training. Lukas et al. (2022) found no surveyed scheme that reliably survived fine-tuning and pruning, and all require proactive insertion: unwatermarked legacy checkpoints cannot be verified.

IPGuard (Cao et al., 2021) extracts inputs near classification boundaries; adversarial frontier stitching (Le Merrer et al., 2020) marks decision surfaces with adversarial examples. CKA (Kornblith et al., 2019) and SVCCA (Raghu et al., 2017) compare representations on a probe dataset. These methods measure functional or representational similarity. Such similarity does not, by itself, certify weight inheritance, although whether a specific method produces false lineage evidence depends on the

model pair and operating threshold. They also require data and forward passes.

Weight cosine, Frobenius distance, and permutation matching (Ainsworth et al., 2023) detect derived checkpoints when the suspect remains close in parameter space, but these are uncalibrated global scalars with no structural account of lineage. HuRef (Zeng et al., 2024) relies on LLM parameter convergence after pretraining. REEF (Zhang et al., 2025) needs probe data or activations. MoTher (Horwitz et al., 2025) recovers model trees but requires a checkpoint collection rather than a single pair. Our fingerprint is data-free, operates on a single pair against a reference-specific empirical null, and provides per-block similarity scores that track partial overlap (Appendix D.2).

Park (2026) observed that correctly paired projections in trained residual blocks produce trace-concentrated products and used this structure to recover projection correspondences. Our work builds on this observation but differs in several ways: (1) we decompose the branch product into a generic identity-aligned component and a checkpoint-specific centered remainder; (2) we use the centered remainder as a cross-checkpoint signature rather than treating trace concentration itself as evidence of ancestry; (3) we construct a symmetric model-level lineage score; (4) we analyze the signature’s algebraic invariance to hidden permutation and reciprocal rescaling; and (5) we evaluate under post-training transformations, function-preserving reparameterization, partial inheritance, and distillation. The prior observation that trained residual blocks produce trace-concentrated products is not our contribution; our contribution is the centering decomposition, the lineage score construction, the invariance analysis, and the checkpoint-level evaluation.

3 Residual Signatures for Lineage Verification

A residual block computes $x_{\ell+1} = x_{\ell} + F_{\ell}(x_{\ell})$, where F_{ℓ} is the residual branch. For a branch with K linear layers $W_1, \dots, W_K$ interleaved with nonlinearities, the *residual branch product* composes the linear factors, dropping nonlinearities:

$$M_{\ell} = W_K W_{K-1} \cdots W_1 \in \mathbb{R}^{d \times d} \quad (1)$$

Individual factors W_k are rectangular and do not share an input–output basis; their composed prod-

Model	L	Pair Acc	AUC
GPT-2 (124M–1.5B)	12–48	100%	1.00
BERT-base	12	100%	0.97
LLaMA-2-7B	32	100%	1.00
Mistral-7B	32	100%	0.96
Qwen2.5-7B	28	80%	0.93
DeepSeek-R1	32	80%	0.93

Table 2: Trace concentration across language model families. Pair accuracy measures correct within-block projection recovery via Hungarian matching on $s(i, j)$. Random-init baselines: $\leq 9\%$. See Figure 5 for accuracy vs. scale.

uct maps the residual stream back into its own coordinate space, so its diagonal entries compare like with like. The exact factorization is architecture-dependent (Appendix A.2, Table 9).

3.1 Trace Concentration in Branch Products

Under the Frobenius inner product $\langle A, B \rangle = \text{tr}(A^\top B)$, the identity matrix and the traceless matrices span orthogonal subspaces of $\mathbb{R}^{d \times d}$. Any branch product therefore splits into a component along the identity, with coefficient $\langle M_\ell, I \rangle / \langle I, I \rangle = \text{tr}(M_\ell) / d$, and an orthogonal traceless remainder:

$$M_\ell = \frac{\text{tr}(M_\ell)}{d} I + E_\ell \quad (2)$$

where $\text{tr}(E_\ell) = 0$. Traces are predominantly negative (Appendix C.2); the score uses magnitude.

The *normalized trace concentration* (Park, 2026) measures this identity alignment:

$$s(M) = \frac{|\text{tr}(M)|}{\|M\|_F}, \quad (3)$$

the fraction of the matrix’s energy along the identity direction, up to $\sqrt{d}$. Training moves energy into the identity component: correctly paired branch products score far above unpaired or untrained ones (Figure 1b). Park (2026) show that the correct-pair score approaches $\sqrt{d}$ as the traceless residual E shrinks, while mismatched projections yield $\mathbb{E}[s] \approx 1/\sqrt{d}$ —a margin that grows with dimension.

This trace concentration is not specific to MLPs. Table 2 shows that the phenomenon appears across six language model families. We extract architecture-aware branch products (Appendix A.2) from public checkpoints and apply Hungarian matching on the score matrix $s(i, j)$. Block-pairing accuracy reaches 80–100% across

all families, against random-initialization baselines of at most 9%. The extraction transfers across GELU (Hendrycks and Gimpel, 2016) and SwiGLU (Shazeer, 2020) activations with no per-family tuning; only the factorization changes. The phenomenon also transfers to vision (ViT, ResNet) and speech (Whisper) architectures (Appendix E). This universality motivates building lineage signatures on the trace-concentrated structure.

3.2 A Coupling-Based Account of Identity Alignment

We propose a gradient-coupling account for the observed identity alignment, though we note this is a plausible mechanism rather than a proven cause.

The skip connection changes what the branch must learn. A plain layer must transmit the entire representation; a residual branch learns only a correction around an identity path that already transmits everything. This suggests a gradient consequence: W_{in} and W_{out} are the two ends of one correction, and the loss reaches both through the same branch. ∇W_{out} depends on the branch input, a function of W_{in} ; ∇W_{in} depends on the upstream gradient, which flows through W_{out} . Their updates may therefore be correlated by construction. In a plain network no such pairing is privileged; every adjacent pair of layers is coupled equally, so no per-block signature can form.

Under this account, the correlation accumulates along the identity direction because the skip means the branch learns a correction, not the full map. Small coordinated corrections to an identity baseline accumulate along the diagonal.

A natural alternative account is dynamical isometry (Pennington et al., 2017): if training enforced near-orthogonal block Jacobians $J = I + J_F$, identity alignment would follow as a byproduct of pressure toward $J \approx I$. This predicts trained Jacobians should be more orthogonal than at initialization.

Empirical evidence. We test this account with four experiments. First, we train depth-24 residual MLPs on CIFAR-10 (Krizhevsky, 2009) under seven initialization schemes. Block-pairing accuracy starts at chance ($\leq 2.1\%$) and reaches 93–100% after training for every converging scheme (Appendix C, Table 11). Orthogonal initialization satisfies dynamical isometry exactly yet yields zero correct pairs at init, confirming the fingerprint requires training.

Second, we compare ResNet-24 against

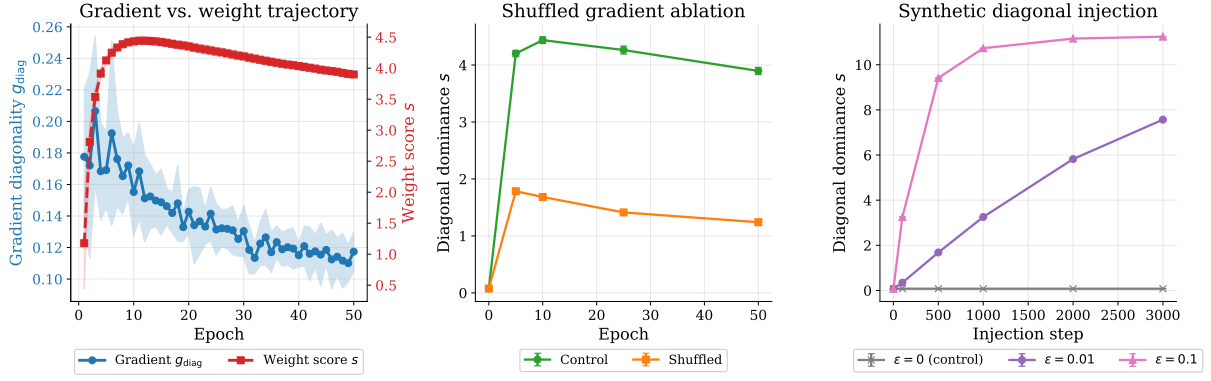


Figure 2: (a) Gradient diagonality g_{diag} (blue) stays flat at ~ 0.15 while the weight score s (orange) rises to ~ 4.0 : individual gradient updates are not diagonal, but their accumulation is. (b) Shuffling ∇W_{out} across blocks (orange) reduces the final score by 68% compared to control (blue), providing evidence that within-block coupling contributes to the signal. (c) Injecting synthetic diagonal updates $\Delta W = -\epsilon \cdot e_i^\top$ directly into weights builds the fingerprint from scratch without backpropagation, showing that coordinated identity-aligned updates can generate the observed structure.

PlainNet-24 (skip connection removed). ResNet-24 reaches 100% pair accuracy; PlainNet-24, trained to similar loss, stays at chance (3%). The skip connection is required.

Third, we measure Jacobian orthogonality across the GPT-2 family. Trained Jacobians are $5\text{--}12\times$ less orthogonal than at initialization (GPT-2-small: 0.297 vs 0.025), inconsistent with the isometry hypothesis (Appendix C.2).

Fourth, we track gradient diagonality during training (Figure 2a). The gradient product g_{diag} stays flat (~ 0.15)—individual updates are not diagonal—yet the weight score climbs to ~ 4.0 . The diagonal structure emerges from accumulated correlated updates. Shuffling gradients across blocks reduces the fingerprint by 68% (Figure 2b); injecting synthetic identity-aligned updates without backpropagation builds it from scratch (Figure 2c).

3.3 Centered Residual Signatures

Trace concentration identifies valid trained pairings, not ancestry: independently trained models exhibit the same identity-aligned component. The verifier operates on the centered traceless remainder E_ℓ , using trace concentration only to gate unreliable branches. The identity component is generic; E_ℓ is checkpoint-specific, surviving weight-preserving transformations but not reinitialization. A direct ablation confirms centering’s value: on the GPT-2 benchmark, uncentered branch-product cosine similarity shows 0.027 spurious similarity between independent models, while centering reduces this to 0.0015—an $18\times$ improvement (Appendix C.3).

For each branch product M_ℓ we retain only the traceless remainder of decomposition (2), normalized to a unit vector:

$$R_\ell = M_\ell - \frac{\text{tr}(M_\ell)}{d} \cdot I, \quad \phi_\ell = \frac{\text{vec}(R_\ell)}{\|\text{vec}(R_\ell)\|_2} \quad (4)$$

$R_\ell = E_\ell$ from (2). The signature ϕ_ℓ carries checkpoint-specific geometry that weight-preserving transformations inherit and independent training cannot reproduce. Because M_ℓ is invariant under hidden permutation and reciprocal rescaling, so is every signature built on it (Section 4.2).

Given reference A and suspect B with L blocks each, we form the similarity matrix $G_{ij} = \langle \phi_i^A, \phi_j^B \rangle$, gated by trace concentration (Appendix B), and recover the block correspondence via the Hungarian algorithm (Kuhn, 1955):

$$\pi^* = \arg \max_{\pi \in \mathcal{P}_L} \sum_i G_{i, \pi(i)} \quad (5)$$

For derived checkpoints preserving block order, π^* should be the identity; recovering it without meta-data confirms that signatures track block identity. Complexity: branch products $O(Ld^2h)$, similarity matrix $O(L^2d^2)$, assignment $O(L^3)$.

3.4 Lineage Score and Verification

Diagonal dominance certifies trained residual structure, not ancestry: a verifier built on the dominance score alone achieves AUROC=0.417 (Appendix D.3, Figure 7). To distinguish lineage, we compare the centered signatures ϕ_ℓ . The lineage score averages signature similarity over aligned

branches:

$$\mathcal{L}(A, B) = \frac{1}{L} \sum_{\ell=1}^L \langle \phi_{\ell}^A, \phi_{\pi^*(\ell)}^B \rangle \quad (6)$$

Each term is a cosine similarity, so $\mathcal{L} \in [-1, 1]$ with $\mathcal{L}(A, A) = 1$. In practice, all observed scores are non-negative: related checkpoints score near 1, unrelated ones near 0, since independently initialized weights produce uncorrelated E_{ℓ} . The score is symmetric.

We calibrate against an empirical null distribution $\mathcal{N}_A = \{\mathcal{L}(A, U_j)\}$ from independently trained models U_j . The protocol returns RELATED if $\mathcal{L}(A, B)$ exceeds the maximum null score, and UNRELATED otherwise. Let $\tau = \max_j \mathcal{L}(A, U_j)$:

$$V(A, B) = \text{RELATED} \Leftrightarrow \mathcal{L}(A, B) > \tau \quad (7)$$

With n null models, this yields a conformal p-value $\leq 1/(n+1)$. The guarantee requires exchangeability: descendants from the same root are not independent samples for calibration purposes—the effective sample size is governed by the number of independent roots, not the number of pairwise comparisons. We report AUROC as the primary metric, which summarizes discrimination across thresholds without requiring specific false-positive rate claims. Compatibility (same L and d) is checked before scoring; the protocol returns INCOMPATIBLE otherwise. Memory: $O(L^2 + d^2)$ beyond the checkpoints.

4 Experiments

4.1 Baseline Comparison

We compare seven lineage detection methods on two controlled benchmarks with known ancestry. For the MLP benchmark, we train 2 root models ($L=16$, $d=48$, input=16) for 120 epochs on synthetic regression tasks. From each root, we derive 15 descendants via fine-tuning on the same task (30 epochs), fine-tuning on a different synthetic target function (30 epochs), weight perturbation ($\sigma=0.02$), pruning (30% sparsity), and quantization (8-bit), with 3 variants each. We also train 8 independent models per root using the same architecture but different random seeds, plus 3 distilled students per root ($T=2.0$, $\alpha_{\text{CE}}=0.5$, 60 epochs) that learn to mimic the root’s outputs. This yields 52 pairs: 30 positive (root $\rightarrow$ descendant) and 22 negative (16 independent + 6 distilled).

We construct a GPT-2 benchmark using $\sim 35\text{M}$ -parameter language models ($L=6$, $d=384$) trained for 3 epochs on TinyStories (Eldan and Li, 2023). We train 8 root models and derive descendants via continued pretraining on TinyStories (1 epoch), LoRA merge ($r=8$, $\alpha=16$, 1 epoch), pruning (30/50/70%), and quantization (8/6-bit). We also train distilled students ($T=2.0$, $\alpha_{\text{CE}}=\alpha_{\text{KL}}=0.5$, 2 epochs). Evaluating on the test-split roots (3 of 8) yields 45 pairs: 21 positive and 24 negative (3 distilled students plus 21 cross-root independent comparisons).

Each (reference, suspect) pair receives a binary label: positive if the suspect inherits weights from the reference, negative otherwise. We report AUROC (threshold-free discrimination) and Gap- Z , the standardized margin between positive and negative score distributions:

$$\text{Gap-}Z = \frac{\mu_+ - \mu_-}{\sqrt{(\sigma_+^2 + \sigma_-^2)/2}} \quad (8)$$

where $\mu_{\pm}$ and $\sigma_{\pm}$ are the means and standard deviations of the positive and negative score distributions. Gap- Z measures effect size: values above +3 indicate non-overlapping distributions. Note that σ_- is estimated from few independent roots (3 in GPT-2), so Gap- Z values should be interpreted as approximate.

Table 3 compares the seven methods across both benchmarks. On MLP, four weight-space methods achieve AUROC=1.0: ours, weight cosine, aligned Frobenius, and singular-value distance. SVCCA also achieves perfect discrimination but requires forward passes through both models. CKA and IPGuard underperform (AUROC=0.83 and 0.70), with IPGuard’s negative Gap- Z indicating it sometimes ranks non-descendants higher than descendants.

The GPT-2 results reveal which methods generalize. Three weight-space methods maintain AUROC=1.0: ours, weight cosine, and aligned Frobenius. Singular-value distance degrades substantially (AUROC=0.73), suggesting sensitivity to the higher-dimensional weight matrices in transformer MLPs. Among activation-based methods, SVCCA remains strong (0.99) while CKA stays weak (0.86). IPGuard improves on GPT-2 (0.91) but still lags behind weight-space approaches.

We validate on public checkpoints (Table 4): comparing LLaMA-2-7B-base against 3 documented derivatives and 7 independently trained

Method	Type	Data-free	MLP (52 pairs)		GPT-2 (45 pairs)	
			AUROC↑	Gap-Z↑	AUROC↑	Gap-Z↑
Centered Residual Signature (ours)	Weight	✓	1.00	+53.0	1.00	+31.0
Weight Cosine	Weight	✓	1.00	+76.3	1.00	+30.7
Aligned Frobenius	Weight	✓	1.00	+72.0	1.00	+3.9
Singular Value Distance	Weight	✓	1.00	+7.9	0.73	+0.4
SVCCA (Raghu et al., 2017)	Activation	✗	1.00	+45.4	0.99	+3.7
CKA (Kornblith et al., 2019)	Activation	✗	0.83	+8.6	0.86	+1.9
IPGuard (Cao et al., 2021)	Decision	✗	0.70	−3.5	0.91	+2.1

Table 3: Lineage detection baselines on MLP and GPT-2 benchmarks. Data-free methods operate directly on weights without probe samples. A reliable fingerprint achieves AUROC near 1.0 with large Gap-Z, indicating clean separation between descendants and non-descendants.

models that share identical architecture (32 layers, $d=4096$, intermediate=11008, SwiGLU). The derivatives produce high scores ($\mathcal{L}$: 0.995, 0.996, 0.336) while all 7 independent models score $|\mathcal{L}| < 5 \times 10^{-5}$ —a $\sim 10,000\times$ gap. This expanded null set ($n=7$) provides stronger calibration than a single independent model and confirms that the method reliably distinguishes true descendants from architectural clones trained from scratch.

Pair (base vs.)	Kind	$\mathcal{L}$	Verdict
Llama-2-7B-chat	RLHF	0.995	REL
Vicuna-7B	instruct	0.996	REL
CodeLlama-7B	code PT	0.336	REL
OpenLLaMA-7B (Geng and Liu, 2023)	scratch	$4e-4$	UNR
OpenLLaMA-7B-v2 (Geng and Liu, 2023)	scratch	$-2e-5$	UNR
Amber (Liu et al., 2023)	scratch	$-3e-5$	UNR
Baichuan-7B (Yang et al., 2023)	scratch	$-5e-5$	UNR
Baichuan2-7B (Yang et al., 2023)	scratch	$-5e-6$	UNR
InternLM-7B (Cai et al., 2024)	scratch	$2e-5$	UNR
Yi-6B (Young et al., 2024)	scratch	$2e-5$	UNR

Table 4: Public checkpoint case study (LLaMA-2 family). Descendants score $\mathcal{L} \geq 0.3$; all 7 independent models with identical architecture score $|\mathcal{L}| < 5 \times 10^{-5}$, confirming the method distinguishes weight ancestry from architectural similarity.

Why do weight-space methods outperform activation and decision-boundary approaches? Activation-based methods like CKA and SVCCA measure representational similarity, which can be high even between independently trained models that learn similar features. Decision-boundary methods like IPGuard measure output agreement, which conflates behavioral similarity with weight ancestry—a distilled student may produce similar outputs without sharing any weights. Weight-space methods directly compare the parameters that define ancestry, avoiding these confounds.

4.2 Robustness Under Function-Preserving Checkpoint Laundering

Table 3 shows that all weight-space baselines achieve strong results on clean benchmarks. We now test a more stringent condition: *function-preserving laundering*, where an adversary disguises a stolen model by rearranging its internal structure—shuffling neurons or rescaling weights—without changing what the model computes. We evaluate all weight-space baselines plus Re-Basin+scale (Ainsworth et al., 2023), which covers alignment via L Hungarian assignments per model pair.

To generate reliably laundered checkpoints, we leverage two function-preserving operations. **Ne-block permutation (P)** reorders hidden units ($\mathbf{W}_{\text{in}} \leftarrow P\mathbf{W}_{\text{in}}$, $\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}}P^\top$); **reciprocal rescaling (D)** scales them ($\mathbf{W}_{\text{in}} \leftarrow D\mathbf{W}_{\text{in}}$, $\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}}D^{-1}$). These are the natural function-preserving symmetries of MLP blocks: P exploits neuron-ordering equivalence; D exploits positive homogeneity of ReLU-family activations. Orthogonal rotation Q of the residual stream ($M \mapsto QMQ^\top$) would also cancel in the branch product, but is not function-preserving for models with learned LayerNorm or RMSNorm parameters (Limitations). Re-Basin+scale can recover alignment under P and D, but at higher computational cost: it solves L Hungarian assignments per pair ($O(Ld^3)$), while our method computes branch products directly ($O(Ld^2h)$). All laundered variants pass a function-preservation gate: the model’s outputs change by at most 10^{-4} .

We take the 52-pair MLP benchmark ($L=16$, $d=48$) and launder six variants: none (baseline), P (random permutation per block), D_m (mild rescaling, log-uniform $[0.5, 2]$), D_s (strong rescaling, log-uniform $[0.1, 10]$), PD (permutation then rescal-

Cond.	Bench	AUROC				
		Ours	Re-Basin	Al. Frob	SVD	W. Cos
P	MLP	1.0	1.0	0.50	1.0	0.86
	GPT-2	1.0	1.0	1.0	0.76	1.0[†]
D _m	MLP	1.0	1.0	1.0	0.0	1.0
D _s	MLP	1.0	1.0	0.0	0.0	1.0
PD	MLP	1.0	1.0	0.0	0.0	0.80
PDFT	MLP	1.0	1.0	0.0	0.0	0.80
Lat.	MLP	0.4	0.8	1.3	2.6	1.5
	GPT-2	5	388	25	1438	50

Table 5: AUROC under function-preserving laundering. P=permutation, D_{m/s}=mild/strong rescaling, Lat.=latency (ms). Our method and Re-Basin maintain AUROC=1.0 across all conditions; raw baselines collapse. Ours is 2× faster on MLP, 76× faster on GPT-2. [†]Weight cosine scores collapse 97% (see Appendix D.1).

ing), and PDFT (PD followed by 5 epochs fine-tuning). We also run P on the GPT-2 benchmark but cannot run D variants because GPT-2 uses GELU activation, which is not positively homogeneous ($\text{GELU}(\alpha x) \neq \alpha \cdot \text{GELU}(x)$), so rescaling is not function-preserving.

Table 5 compares methods under laundering. Our signature and Re-Basin+scale both maintain AUROC=1.0 across all conditions, while raw baselines collapse: aligned Frobenius drops to 0.50 under permutation on MLP and 0.0 under strong rescaling; SVD fails rescaling entirely; weight cosine degrades to 0.80 under PD. Aligned Frobenius retains AUROC=1.0 on GPT-2 under permutation, but this is an artifact of the block-level Hungarian assignment: with only 6 blocks (vs. 16 in MLP), per-block norm statistics remain distinctive enough to recover correct layer correspondence even when within-block similarity is destroyed. The Gap-Z margin still collapses from +72 to +3.9 (Appendix D.1). The key differentiator is computational cost. Re-Basin+scale solves L Hungarian assignments per pair ($O(Ld^3)$ complexity), while our method computes branch products directly ($O(Ld^2h)$). This yields 2× speedup on MLP (0.4ms vs 0.8ms) and 76× speedup on GPT-2 (5ms vs 388ms; see Appendix A.1 for detailed statistics). Our method achieves invariance algebraically, with no alignment search.

AUROC alone understates the difference between methods. Appendix D.1 reports Gap-Z scores showing that weight cosine’s margin collapses catastrophically under permutation: descendant scores drop from ~ 0.99 to ~ 0.03 , nearly over-

Base	Suspect	Relation	$\Delta W.\text{cos}$
LLaMA-2 7B	chat	derivative	0.993
LLaMA-3 8B	Inst	derivative	0.949
Mistral 7B	Inst	derivative	0.934
Qwen2.5 7B	Inst	derivative	0.948
LLaMA-2 7B	OpenLLaMA	independent	(0.089) [†]
LLaMA-3 8B	Mistral 7B	independent	(0.158) [†]

Table 6: Permutation laundering on public language model derivatives. $\Delta W.\text{cos}$ = score before – score after for derivatives. Our lineage score $\Delta \mathcal{L} < 10^{-7}$ (invariant); weight cosine loses >93% of its signal in all derivative cases. [†]Raw W.cos score (no laundering applied; shown for baseline comparison).

lapping with non-descendants. The ranking barely survives, but one additional perturbation would break it. Our signature maintains Gap-Z $\approx +53$ across all conditions.

We replicate on public checkpoints from four 7B-scale model families (Table 6): LLaMA-2 (Touvron et al., 2023), LLaMA-3 (Grattafiori et al., 2024), Mistral (Jiang et al., 2023), and Qwen2.5 (Yang et al., 2024). For each base model and its fine-tuned derivative, we shuffle hidden units in the derivative’s MLP layers and measure how much the lineage scores change. Our score is unaffected ($\Delta \mathcal{L} < 10^{-7}$); weight cosine loses $\sim 95\%$ of its signal.

4.3 Robustness to Post-Training Techniques

Section 4.1 showed that all weight-space baselines achieve AUROC=1.0 on clean benchmarks. Here we stress-test how robust our fingerprint is to common post-training transformations that modify weights while preserving model behavior.

For MLPs, we train 3 root models ($L=24, d=48$) for 120 epochs on synthetic regression tasks. Per root we generate 25 descendants via fine-tuning, weight perturbation, pruning (10–85% sparsity), and quantization (16–256 levels), plus 28 unrelated checkpoints (independent same-task, different-task, random-init, and distilled students). This yields 159 pairs: 75 positive and 84 negative. Figure 1c visualizes MLP results: descendants cluster near $\mathcal{L}=1$ while non-descendants fall below the calibrated threshold. Even at 85% sparsity, the minimum descendant score (0.58) remains $\sim 3\times$ above the maximum non-descendant score (0.20).

Table 7 reports GPT-2 lineage scores on the same benchmark (Section 4.1), broken down by transformation type. Quantization and LoRA preserve the

Transformation	n	Mean $\mathcal{L}$	Min $\mathcal{L}$
<i>Descendants</i>			
Quantized (INT8/6)	6	0.999	0.999
LoRA merge (rank-8)	3	0.998	0.996
Fine-tuned (1 epoch)	3	0.980	0.980
Pruned (30–70%)	9	0.937	0.855
<i>Non-descendants</i>			
Distilled student	3	0.002	0.001
Independent	21	0.003	0.000

Table 7: Lineage scores under post-training transformations (GPT-2 benchmark from Section 4.1, 3 test roots). The minimum descendant score (0.855 at 70% sparsity) exceeds the maximum non-descendant score (0.004) by $200\times$.

signal almost perfectly ($\mathcal{L} \geq 0.996$); fine-tuning and pruning degrade it but remain well above the null ($\mathcal{L} \geq 0.855$). Both distilled students and independently trained models score near zero. The $200\times$ gap between minimum descendant and maximum non-descendant scores yields AUROC=1.0 with perfect separation.

Behavioral similarity does not imply weight ancestry. A key question is whether output agreement implies weight-level lineage. Distillation increases teacher–student agreement modestly (+1.4 points top-1 accuracy over independent training, 79.0% vs 77.6%) while producing no weight ancestry signal ($\mathcal{L} \approx 0.002$). Models trained on the same architecture and corpus already exhibit high agreement; distillation raises this slightly, yet neither independently trained nor distilled models show weight lineage. This confirms that our method detects shared parameters, not behavioral mimicry.

5 Conclusion

We introduced centered residual signatures for data-free verification of weight-level model lineage. Trace concentration is shared by independently trained residual networks and therefore cannot itself establish ancestry; the checkpoint-specific signal lies in the traceless remainder. Comparing these remainders across aligned blocks yields a symmetric score algebraically invariant to hidden-unit permutation and reciprocal rescaling.

On controlled residual-MLP and GPT-2 benchmarks, the score separates descendants from independent and distilled models with AUROC=1.0. Raw weight-space baselines perform equally well on clean checkpoints but lose margin or fail under function-preserving laundering; our method re-

mains unchanged and is $76\times$ faster than alignment-based recovery on GPT-2. The prerequisite projection-pairing signal appears across six language model families (and transfers to vision and speech architectures; Appendix E). A public LLaMA-2 case study with 7 independently trained architectural clones shows a $\sim 10,000\times$ score gap between descendants and unrelated models, providing strong null calibration across diverse training setups. Centered residual signatures provide provenance evidence for compatible white-box residual models, not proof of ownership or direction of descent. Deployment requires public-checkpoint validation and evaluation under aggressive modifications and adaptive attacks.

Ethics Statement

We present a method for detecting shared weight ancestry between neural network checkpoints. Potential applications include model provenance verification and supply-chain auditing. We acknowledge several ethical considerations:

Limitations of evidence. The method produces a similarity score, not a legal determination. Scores consistent with documented relationships do not independently prove provenance, ownership, or wrongdoing. Results should be combined with metadata, release records, and human review before drawing conclusions about model origins.

False confidence risk. Limited calibration data (few independent roots) may produce misleadingly precise-seeming thresholds. Users should understand that formal hypothesis testing requires sufficient exchangeable samples, which may not be available for rare architectures.

Privacy considerations. The method could potentially be used to identify undocumented model relationships, which may conflict with developers’ reasonable expectations of anonymity. The white-box access requirement limits covert use, but authorized access does not automatically imply consent to provenance analysis.

Dual use. The same method that enables legitimate supply-chain auditing could be misused for unfounded infringement claims or competitive intelligence. We encourage responsible deployment with appropriate evidentiary standards.

Limitations

Scope and architecture constraints. Our method needs white-box access to both checkpoints, so

API-only models cannot be verified. It also only works for residual architectures: plain feedforward networks, RNNs, and state-space models are out of scope. Comparisons are further limited to models with matching depth and hidden dimension, so cross-architecture verification (e.g., LLaMA vs. GPT-2) is not supported.

Lineage determination. The lineage score is symmetric, so without external metadata we cannot tell which checkpoint is the ancestor. The score decays monotonically with genealogical distance, but the method compares single pairs and does not reconstruct multi-hop ancestry or family trees. Under partial inheritance (layer grafts, linear merges), the score reflects the fraction of shared blocks but not which blocks were inherited.

Signal degradation under transformation. The fingerprint survives common post-training modifications, but weakens as transformations grow more aggressive. Heavy pruning drops the signal to $\mathcal{L} = 0.58$ at 85% sparsity, and extensive continued pretraining (CodeLlama) lowers it to $\mathcal{L} = 0.336$, though both remain detectable above the null. In the evaluated MLP suppression attack, reaching the null threshold ($\mathcal{L} \approx 0.084$) costs +1.5% utility loss; driving the score reliably below null costs +12%. This result does not establish robustness to other attacks, architectures, utility measures, or optimization budgets.

Residual-stream rotation. The signature is invariant to hidden-unit permutation and reciprocal rescaling within MLP blocks, but not to orthogonal rotation Q of the residual stream: $M \mapsto QMQ^\top$ preserves trace but transforms the centered remainder, destroying cross-checkpoint similarity. For LayerNorm architectures (GPT-2, BERT), such rotation is not function-preserving because learned element-wise γ and β parameters do not commute with arbitrary Q . For RMSNorm architectures with learned per-dimension scale (LLaMA, Mistral), the same obstruction applies unless γ is constant across dimensions, which is not standard practice. An architecture using only unparameterized normalization (e.g., pure RMS without learned scale) would be vulnerable to Q -rotation laundering; we are not aware of widely deployed models in this category.

Calibration and validation. Verification requires calibrating a null distribution from independently trained models of the same architecture, and the threshold may need adjustment across architecture families. Our controlled benchmarks demonstrate perfect AUROC discrimination, but the ef-

fective sample sizes for formal hypothesis testing are limited: the GPT-2-style benchmark has 8 roots total. The LLaMA-2 case study now includes 7 independent models (OpenLLaMA v1/v2, Amber, Baichuan v1/v2, InternLM, Yi), providing stronger calibration for that architecture family, but other families remain undertested. Real-world validation covers a limited set of public checkpoints, so broader ecosystem coverage is untested. SwiGLU architectures (Qwen2.5, DeepSeek-R1) reach only 80% block-pairing accuracy and need joint factorization to recover sub-paths reliably.

Use of AI Assistants

The authors bear full responsibility for all scientific claims, experimental results, and analysis in this work. During manuscript preparation, we used Claude to support literature searches, experimental design, code development, and the drafting and editing of text. All AI-assisted material was reviewed, verified, and revised by the authors.

References

- Yossi Adi, Carsten Baum, Moustapha Cisse, Benny Pinkas, and Joseph Keshet. 2018. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX Security Symposium*, pages 1615–1631.
- Samuel K Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. 2023. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*.
- Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, Xun Chen, et al. 2024. InternLM2 technical report. *arXiv preprint arXiv:2403.17297*.
- Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2021. Ipguard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary. In *AsiaCCS*, pages 14–25.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob

782	Uszkoreit, and Neil Houlsby. 2021. An image	Alex Krizhevsky. 2009. Learning multiple layers of fea-	836
783	is worth 16x16 words: Transformers for image	tures from tiny images. Technical report, University	837
784	recognition at scale. In <i>International Conference on</i>	of Toronto.	838
785	<i>Learning Representations</i> .		
786	Ronen Eldan and Yuanzhi Li. 2023. TinyStories: How	Harold W Kuhn. 1955. The Hungarian method for	839
787	small can language models be and still speak coherent	the assignment problem. <i>Naval Research Logistics</i>	840
788	English? <i>arXiv preprint arXiv:2305.07759</i> .	<i>Quarterly</i> , 2(1-2):83–97.	841
789	Xinyang Geng and Hao Liu. 2023. OpenLLaMA: An	Erwan Le Merrer, Patrick Perez, and Gilles Trédan.	842
790	open reproduction of LLaMA.	2020. Adversarial frontier stitching for remote neu-	843
791	Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri,	ral network watermarking. <i>Neural Computing and</i>	844
792	Abhinav Pandey, Abhishek Kadian, Ahmad Al-	<i>Applications</i> , 32(13):9233–9244.	845
793	Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten,	Zhengzhong Liu, Aurick Qiao, Willie Neiswanger,	846
794	et al. 2024. The Llama 3 herd of models. <i>arXiv</i>	Hongyi Wang, Bowen Tan, Tianhua Tao, Junbo Li,	847
795	<i>preprint arXiv:2407.21783</i> .	Yuqi Wang, Suqi Sun, Omkar Panber, et al. 2023.	848
796	Song Han, Huizi Mao, and William J. Dally. 2016. Deep	LLM360: Towards fully transparent open-source	849
797	compression: Compressing deep neural networks	LLMs. <i>arXiv preprint arXiv:2312.06550</i> .	850
798	with pruning, trained quantization and huffman cod-	Nils Lukas, Edward Jiang, Xinda Li, and Florian Ker-	851
799	ing. In <i>International Conference on Learning Repre-</i>	schbaum. 2022. Sok: How robust is image clas-	852
800	<i>sentations</i> .	sification deep neural network watermarking? In	853
801	Song Han, Jeff Pool, John Tran, and William J. Dally.	<i>2022 IEEE Symposium on Security and Privacy (SP)</i> ,	854
802	2015. Learning both weights and connections for	pages 787–804. IEEE.	855
803	efficient neural networks. In <i>Advances in Neural</i>	Margaret Mitchell, Simone Wu, Andrew Zaldivar,	856
804	<i>Information Processing Systems</i> .	Parker Barnes, Lucy Vasserman, Ben Hutchinson,	857
805	Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian	Elena Spitzer, Inioluwa Deborah Raji, and Timnit	858
806	Sun. 2016. Deep residual learning for image recogni-	Gebbru. 2019. Model cards for model reporting. In	859
807	tion. In <i>IEEE Conference on Computer Vision and</i>	<i>Proceedings of the Conference on Fairness, Account-</i>	860
808	<i>Pattern Recognition</i> , pages 770–778.	<i>ability, and Transparency</i> , pages 220–229.	861
809	Dan Hendrycks and Kevin Gimpel. 2016. Gaus-	Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida,	862
810	sian error linear units (GELUs). <i>arXiv preprint</i>	Carroll L. Wainwright, Pamela Mishkin, Chong	863
811	<i>arXiv:1606.08415</i> .	Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray,	864
812	Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2025.	et al. 2022. Training language models to follow in-	865
813	Unsupervised model tree heritage recovery. In <i>In-</i>	structions with human feedback. In <i>Advances in</i>	866
814	<i>ternational Conference on Learning Representations</i> ,	<i>Neural Information Processing Systems</i> .	867
815	pages 47900–47918.	Hyunwoo Park. 2026. I dropped a neural net . <i>Preprint</i> ,	868
816	Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Worts-	<i>arXiv:2602.19845</i> .	869
817	man, Suchin Gururangan, Ludwig Schmidt, Han-	Jeffrey Pennington, Samuel Schoenholz, and Surya Gan-	870
818	naneh Hajishirzi, and Ali Farhadi. 2023. Editing	guli. 2017. Resurrecting the sigmoid in deep learning	871
819	models with task arithmetic. In <i>International Confer-</i>	through dynamical isometry: theory and practice. In	872
820	<i>ence on Learning Representations</i> .	<i>Advances in Neural Information Processing Systems</i> ,	873
821	Hengrui Jia, Mohammad Yaghini, Christopher A.	volume 30.	874
822	Choquette-Choo, Natalie Dullerud, Anvith Thudi,	Alec Radford, Jong Wook Kim, Tao Xu, Greg Brock-	875
823	Varun Chandrasekaran, and Nicolas Papernot. 2021.	man, Christine McLeavey, and Ilya Sutskever. 2023.	876
824	Proof-of-learning: Definitions and practice. In <i>IEEE</i>	Robust speech recognition via large-scale weak su-	877
825	<i>Symposium on Security and Privacy</i> , pages 1039–	pervision. In <i>Proceedings of ICML</i> .	878
826	1056.	Maithra Raghu, Justin Gilmer, Jason Yosinski, and	879
827	Albert Q. Jiang, Alexandre Sablayrolles, Arthur Men-	Jascha Sohl-Dickstein. 2017. SVCCA: Singular vec-	880
828	sch, Chris Bamford, Devendra Singh Chaplot, Diego	tor canonical correlation analysis for deep learning	881
829	de las Casas, Florian Bressand, Gianna Lengyel, Guil-	dynamics and interpretability. In <i>Advances in Neural</i>	882
830	laume Lample, Lucile Saulnier, et al. 2023. Mistral	<i>Information Processing Systems</i> .	883
831	7B. <i>arXiv preprint arXiv:2310.06825</i> .	Noam Shazeer. 2020. GLU variants improve trans-	884
832	Simon Kornblith, Mohammad Norouzi, Honglak Lee,	former. <i>arXiv preprint arXiv:2002.05202</i> .	885
833	and Geoffrey Hinton. 2019. Similarity of neural	Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier	886
834	network representations revisited. In <i>International</i>	Martinet, Marie-Anne Lachaux, Timothée Lacroix,	887
835	<i>Conference on Machine Learning</i> , pages 3519–3529.	Baptiste Rozière, Naman Goyal, Eric Hambro,	888
		Faisal Azhar, et al. 2023. Llama: Open and effi-	889
		cient foundation language models. <i>arXiv preprint</i>	890
		<i>arXiv:2302.13971</i> .	891

Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. 2017. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*, pages 269–277.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Rober, Ari Morcos, Hannaneh Hajishirzi, Ali Farhadi, Simon Kornblith, and Ludwig Schmidt. 2022. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998.

Aiyuan Yang, Bin Xiao, Bingning Liu, et al. 2023. Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305*.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Alex Young, Bei Chen, Chao Li, et al. 2024. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*.

Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. 2024. HuRef: HUMAN-REAdable fingerprint for large language models. In *Advances in Neural Information Processing Systems*, volume 37.

Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. 2025. REEF: Representation encoding fingerprints for large language models. In *International Conference on Learning Representations*, pages 48092–48117.

A Reproducibility Statement

All results in this paper are produced by deterministic scripts with fixed seeds.¹ The 52-pair MLP lineage benchmark (Tables 10, 3) uses depth-16 MLPs (in_dim=16, hidden=48) and is constructed from 2 trained reference checkpoints, each paired with 5 related-checkpoint kinds $\times$ 3 derivative seeds = 30 truly-related pairs (fine-tune, fine-tune to a new target, $\sigma_{\text{rel}} \in [0.01, 0.15]$ Gaussian noise, 10–85% magnitude pruning, and 16–256-level uniform quantization), plus 16 different-seed same-task pairs and 6 distilled-student pairs as unrelated checkpoints. The expanded 159-pair set (Figure 1c) uses 3 depth-24 MLP reference models: each reference is paired with 25 related checkpoints (5 transformation types $\times$ 5 variations) and 28 unrelated checkpoints (15 independent same-task + 5 independent different-task + 5 random-init + 3

¹Software attached as supplementary material in submission.

distilled), yielding $3 \times 25 = 75$ related pairs and $3 \times 28 = 84$ unrelated pairs ($75 + 84 = 159$ total). Note that these 84 unrelated pairs share only 3 reference model roots; the effective sample size for null calibration is limited by the number of independent roots, not the number of pairs.

The GPT-2-Small-Lite benchmark (Table 7) uses 8 transformer roots with GPT-2 architecture ($L=6$ layers, $d=384$, $d_{\text{ff}}=1536$, 6 heads, $\sim 35\text{M}$ parameters), trained on TinyStories (Eldan and Li, 2023) (50K samples, 3 epochs, batch size 8, lr=3e-4, cosine schedule, fp16, gradient checkpointing). Roots are allocated to calibration ($n=2$), development ($n=3$), and test ($n=3$) splits; Table 7 reports results only on the 3 held-out test roots (21 descendant pairs, 3 distilled pairs, 21 cross-root pairs = 45 total). Each root generates 7 descendants: 1 fine-tuned (1 epoch continued pretraining, lr=1e-4), 1 LoRA-merged (rank-8, $\alpha=16$, 1 epoch, lr=1e-4), 3 pruned (magnitude pruning at 30/50/70% sparsity), and 2 quantized (uniform quantization at 256/64 levels, i.e., INT8/INT6). Additionally, 8 distilled students (one per root) are trained from scratch using knowledge distillation ($T=2.0$, $\alpha_{\text{CE}}=0.5$, $\alpha_{\text{KL}}=0.5$, 2 epochs). Quality metrics for distilled students include top-1 agreement, KL divergence, and perplexity ratio, computed on a held-out validation set.

Real-architecture pairing (Table 2) loads public HuggingFace checkpoints. Code, configuration files, baseline implementations (aligned Frobenius, singular-value distance, weight cosine, CKA, SVCCA, IPGuard regression analogue), the per-pair score JSONs, and the plotting scripts will be released upon acceptance.

Hardware and software. All experiments were run on a single NVIDIA L4 GPU (24GB VRAM) with an AMD EPYC 7R13 CPU. Software: PyTorch, NumPy, SciPy, and Hugging Face Transformers. Global seeds (`torch.manual_seed(0)`, `np.random.seed(0)`) are set at the start of each benchmark script; per-model seeds are derived deterministically from root and variant indices.

Data sources. TinyStories (Eldan and Li, 2023) is loaded via Hugging Face Datasets (roneneldan/TinyStories). CIFAR-10 (Krizhevsky, 2009) uses `torchvision.datasets`. Public checkpoints (GPT-2, LLaMA-2, Mistral, Qwen) are loaded from their official Hugging Face repositories.

A.1 Latency Statistics

Table 8 reports per-method latency on the GPT-2 benchmark (20 pairs, GPU). Our method is $76\times$ faster than Re-Basin+scale and $283\times$ faster than SVD, while achieving identical AUROC under laundering.

Method	Mean	Std	Min	Max
Centered Residual Sig. (ours)	5.1	2.3	4.4	15.1
Re-Basin+scale	387.9	10.9	369.6	424.6
Aligned Frobenius	25.0	1.3	23.6	29.2
Singular Value Distance	1438.2	35.5	1417.7	1585.9
Weight Cosine	50.1	7.7	46.2	83.4

Table 8: Per-method latency (ms) on GPT-2 benchmark. Our method achieves $76\times$ speedup over Re-Basin+scale, the only other method maintaining AUROC=1.0 under laundering.

A.2 Architecture-Aware Factorization

Table 9 lists the architecture-aware residual branch products used throughout this work.

On ImageNet-pretrained ResNet-50/101/152 (torchvision), we compare naive two-layer factorization (W_3W_1 , skipping the middle conv) against the correct three-layer product ($W_3W_2W_1$) on layer3 bottleneck blocks ($n=5/22/35$ blocks respectively). Naive W_3W_1 achieves chance-level accuracy; the correct triple product recovers:

- ResNet-50/layer3 ($n=5$): 100%, AUC 1.000
- ResNet-101/layer3 ($n=22$): 100%, AUC 0.995
- ResNet-152/layer3 ($n=35$): 91%, AUC 0.964

B Verification Protocol Details

B.1 Gated Branch Score

Branch similarity is reliable only when both branches exhibit strong trace concentration. We gate the cosine similarity:

$$G(\phi_\ell^A, \phi_m^B) = \langle \phi_\ell^A, \phi_m^B \rangle \cdot \min\left(\frac{s(M_\ell^A)}{\tau_s}, \frac{s(M_m^B)}{\tau_s}, 1\right) \quad (9)$$

where τ_s is the minimum trace concentration across reference branches.

B.2 Statistical Calibration

We calibrate against a null distribution $\mathcal{N}_A = \{\mathcal{L}(A, U_j) : U_j \notin \mathcal{D}(A)\}$ of scores between reference A and independently trained models. Given n null scores, the empirical threshold $\tau =$

$\max_j \mathcal{L}(A, U_j)$ yields a conformal p-value:

$$p(A, B) = \frac{|\{j : \mathcal{L}(A, U_j) \geq \mathcal{L}(A, B)\}| + 1}{n + 1} \quad (10)$$

With n null models, the minimum achievable p-value is $1/(n + 1)$. We therefore report AUROC as the primary metric, which aggregates discrimination across all thresholds without requiring specific FPR claims.

Calibration power by benchmark. The MLP benchmark has 84 unrelated pairs from 3 independent roots; the effective sample size for calibration is 3 roots, not 84 pairs, since descendants from the same root are not independent samples. The GPT-2-Small-Lite benchmark uses 8 roots total, with 3 held out for testing; each test root is compared against 7 independent roots, but the effective sample size for formal hypothesis testing remains limited. We report AUROC as a discrimination summary rather than threshold-based verification claims with specific false-positive rate guarantees. The LLaMA-2 case study includes 7 independently trained architectural clones (OpenLLaMA v1/v2, Amber, Baichuan v1/v2, InternLM, Yi), all with identical architecture (32 layers, $d=4096$, intermediate=11008). All 7 score $|\mathcal{L}| < 5 \times 10^{-5}$ against the LLaMA-2-7B reference, while descendants score $\mathcal{L} \geq 0.3$ —a $\sim 10,000\times$ gap that provides meaningful null calibration for this architecture family.

C Mechanism Validation

C.1 Initialization Ablation

Table 11 shows pair accuracy before and after training across seven initialization schemes on depth-24 residual MLPs (3 seeds, 200 epochs). All schemes show chance-level accuracy ($\leq 2.1\%$) at initialization, rising to 93–100% after training. Orthogonal initialization provides dynamical isometry at init yet shows zero correct pairs before training.

C.2 GPT-2 Scaling, Trace, and Jacobian Analysis

Table 12 reports the normalized Jacobian deviation $\delta_J^{\text{norm}} = \|J^\top J / \|J\|_F^2 - I/d\|_F$ across GPT-2 scales. This metric is scale-invariant: any scaled orthogonal matrix scores 0, and larger values indicate non-uniform singular values.

Figure 3 shows block pairing score matrices $s(i, j)$ for GPT-2 models from 124M to 1.5B parameters.

Architecture	Residual branch $F(x)$	Product M
Transformer MLP / BasicBlock	$W_2\sigma(W_1x)$	W_2W_1
Attention V/O path	$W_O\text{Attn}(x)W_Vx$	W_OW_V
Attention Q/K path	bilinear $x^\top W_QW_K^\top x$	$W_QW_K^\top$
Bottleneck ResNet	$W_3\sigma(W_2\sigma(W_1x))$	$W_3W_2W_1$
SwiGLU MLP	$W_{\text{down}}[\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x]$	$W_{\text{down}}W_{\text{up}}$

Table 9: Architecture-aware residual branch products. Each M composes the linear maps of one branch (dropping nonlinearities) so it maps the residual stream to itself. Factorization must match the architecture; incomplete products destroy the signal.

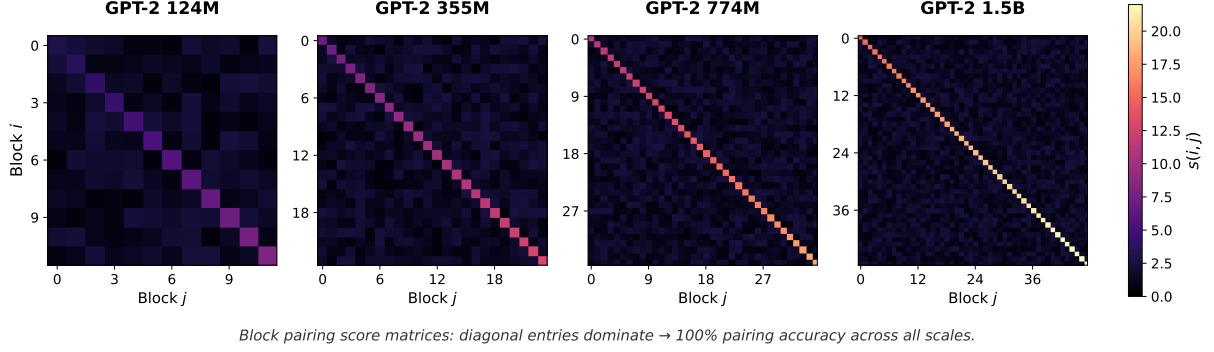


Figure 3: Block pairing score matrices $s(i, j)$ for GPT-2 models from 124M to 1.5B parameters. Diagonal entries dominate, yielding 100% pairing accuracy across all scales.

Transformation	Mean $\mathcal{L}$	Min $\mathcal{L}$	$> \max(\text{null})$
Fine-tune / Quant / Noise	0.99	0.97	45/45
Fine-tune (diff. target)	0.94	0.84	15/15
Pruning (10–85%)	0.81	0.58	15/15
Independent / Distilled	0.08	0.01	-

Table 10: Lineage score survival under post-training modification ($n=75$ related, $n=84$ unrelated). All related checkpoints exceed the maximum null score ($\max \mathcal{L}_{\text{null}} = 0.20$); AUROC=1.000.

Init scheme	Untrained	Trained	AUROC
Orthogonal	0.0%	100%	0.947
Kaiming-normal	2.1%	97%	0.981
Kaiming-uniform	2.1%	98.6%	0.98
Xavier-normal	2.1%	100%	0.990
Xavier-uniform	2.1%	100%	0.99
Uniform	2.1%	93.8%	—
Gaussian $\sigma=0.02$	2.1%	13%	0.671

Table 11: Initialization ablation on depth-24 residual MLPs. All schemes show chance-level accuracy before training. The Gaussian $\sigma=0.02$ case fails because blocks collapse to near-zero contribution.

Figure 4 shows the trace values $\text{tr}(W_{\text{out}}W_{\text{in}})$ per block across all four GPT-2 variants.

Figure 5 compares block pairing accuracy across GPT-2 scales.

C.3 Centering Ablation

The centering step removes the identity-aligned component shared by all trained models. We verify this materially improves discrimination by directly comparing cosine similarity on uncentered branch products $\cos(\text{vec}(M_\ell^A), \text{vec}(M_\ell^B))$ versus centered residuals $\cos(\text{vec}(R_\ell^A), \text{vec}(R_\ell^B))$ where $R_\ell = M_\ell - \frac{\text{tr}(M_\ell)}{d}I$.

Table 13 shows results on the GPT-2 benchmark (27 pairs from 3 test roots). Both methods achieve AUROC=1.0, but centering reduces unrelated-pair similarity by $\sim 50\times$: from 0.019 (uncentered) to

0.0004 (centered). For independent roots specifically, uncentered similarity averages 0.027 while centered similarity averages 0.0015—an $18\times$ reduction.

The identity energy fraction $|\text{tr}(M)|^2/\|M\|_F^2$ averages 9.4 across all layers, confirming that the identity component carries substantial matrix energy. Independent models share this generic identity-aligned structure from training dynamics; centering removes it, revealing the checkpoint-specific remainder that differs between unrelated models.

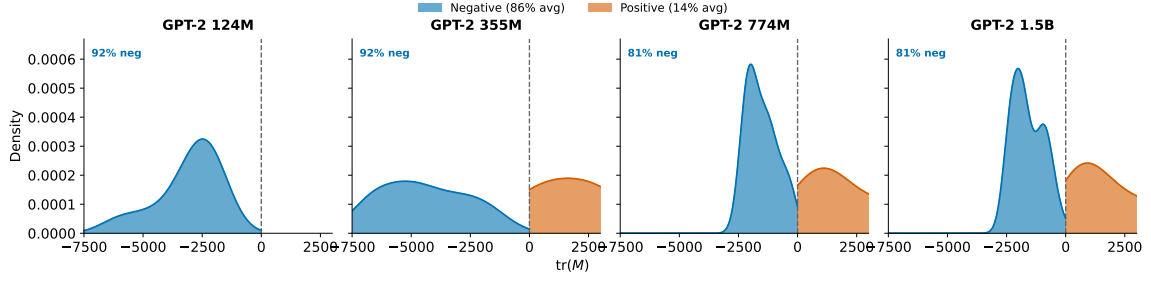


Figure 4: Distribution of trace values $\text{tr}(W_{\text{out}} W_{\text{in}})$ across GPT-2 scales. Blue: negative trace (86% average); orange: positive trace (14% average). The strong skew toward negative values is consistent with the identity-alignment mechanistic account.

Model	d	Pretrained	Rand-init	Ratio
GPT-2-small	768	0.297	0.025	$11.7\times$
GPT-2-medium	1024	0.242	0.026	$9.4\times$
GPT-2-large	1280	0.155	0.025	$6.2\times$
GPT-2-XL	1600	0.122	0.024	$5.1\times$

Table 12: Jacobian orthogonality (δ_J^{norm}) across GPT-2 scales. Pretrained models are 5–12 $\times$ less orthogonal than at random init, inconsistent with the hypothesis that trace concentration results from blocks approaching isometry.

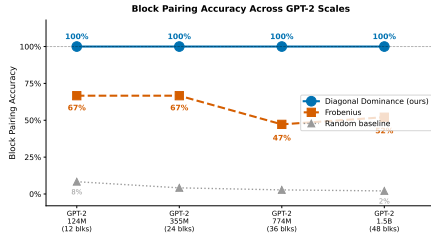


Figure 5: Block pairing accuracy as model size increases. Normalized trace pairing (blue) maintains 100% projection-pair accuracy; Frobenius matching (orange) degrades from 67% to 52%; random baseline (gray) drops from 8% to 2% as the number of blocks increases.

D Extended Benchmarks

D.1 Laundering Experiment Gap-Z Scores

Table 14 reports the same laundering experiment as Table 5 but with Gap-Z instead of AUROC. Gap-Z measures the standardized separation between descendant and non-descendant score distributions; larger values indicate more robust margins.

The key finding: weight cosine’s Gap-Z collapses from +76.3 (none) to +2.1 (P) and +1.8 (PD). Despite preserving AUROC=0.80–0.86, the descendant and non-descendant distributions nearly overlap. One additional perturbation would likely break the ranking entirely. Our signature maintains

Method	AUC	Gap-Z	Unrel. μ	Margin
Uncentered $\text{vec}(M_\ell)$	1.0	+28.7	0.019	0.84
Centered $\text{vec}(R_\ell)$	1.0	+29.2	0.0004	0.86

Table 13: Centering ablation on the GPT-2 benchmark. Both methods perfectly separate related from unrelated pairs, but centering reduces spurious similarity by $\sim 50\times$.

		Gap-Z				
Cond.	Bench	Ours	Re-Basin	Al. Frob	SVD	W. Cos
none	MLP	+53.0	+48.2	+72.0	+7.9	+76.3
P	MLP	+53.0	+48.2	−0.1	+7.9	+2.1
P	GPT-2	+31.0	+29.5	+3.9	+0.4	+0.8
D_m	MLP	+53.0	+48.2	+72.0	−5.2	+76.3
D_s	MLP	+53.0	+48.2	−4.8	−5.2	+76.3
PD	MLP	+53.0	+48.2	−4.8	−5.2	+1.8
PDFT	MLP	+49.1	+45.0	−4.8	−5.2	+1.8

Table 14: Gap-Z under function-preserving laundering (same experiment as Table 5). Weight cosine collapses from +76.3 to +2.1 under permutation and +1.8 under PD—the margin is razor-thin despite AUROC=0.80–0.86. Our signature maintains Gap-Z $\approx +53$ across all conditions.

Gap-Z $\approx +53$ across all conditions, indicating stable separation regardless of laundering.

D.2 Harder Regime: Layer Grafts and Linear Merges

We construct 40 harder pairs from depth-16 MLPs (same setup as Table 3): (i) **layer grafts** copy $K \in \{0, 4, 8, 12, 16\}$ of 16 blocks from reference into an independent donor (related if $K \geq 8$); (ii) **linear merges** compute $w \cdot \text{ref} + (1-w) \cdot \text{donor}$ with $w \in \{0, 0.25, 0.5, 0.75, 1.0\}$ (related if $w \geq 0.5$). We use 2 references and 2 donors per reference. AUROC measures binary related-vs-unrelated detection; Spearman correlation measures whether a method tracks partial overlap monotonically (K/L

or w) rather than just clearing a threshold. Table 15 summarizes the 40-pair benchmark.

Method	AUROC		Spearman ρ	
	Graft	Merge	Graft	Merge
Ours	0.98	1.00	+ .96	+ .96
Aligned Frob.	1.00	1.00	+ .99	+ .99
Weight cos.	1.00	1.00	+ .98	+ .96
SVD dist.	1.00	0.45	+ .99	+ .23
SVCCA	1.00	1.00	+ .97	+ .99
CKA	0.81	0.78	+ .73	+ .75
IPGuard	0.05	0.45	− .83	+ .22

Table 15: Harder-regime benchmark on layer grafts and linear merges (40 pairs). Our method tracks partial overlap monotonically ($\rho \geq 0.96$). CKA degrades; IPGuard collapses on grafts; SVD fails merges.

D.3 CIFAR-10 ResNet-18 Benchmark

We train two CIFAR-10 ResNet-18 references and three independents. Each reference yields 11 related checkpoints (noise 1–10%, pruning 20–80%, quantization 16–256 levels). Results: AUROC=1.000; all related checkpoints exceed the maximum null score. Related checkpoint scores: $\mathcal{L} \in [0.695, 1.000]$; unrelated baseline: $\mathcal{L}_{\max} = 0.004$.

D.4 GPT-2-Small-Lite Benchmark

Figure 6 visualizes the GPT-2-Small-Lite benchmark results from Section 4.3. Panel (a) shows lineage score distributions by transformation type: descendants (quantized, LoRA, fine-tuned, pruned) score high while non-descendants (distilled students, independent roots) score near zero. The minimum descendant score (0.855) exceeds the maximum non-descendant score (0.004) by 200 $\times$. Panel (b) shows that distillation modestly improves teacher–student agreement (79% top-1, +1.4 points over independent training) yet leaves the weight-lineage score near the independent-root range ($\mathcal{L} \approx 0.002$). This illustrates the conceptual distinction: behavioral similarity does not imply weight ancestry.

D.5 ROC for Lineage Verification

Figure 7 shows that trace concentration alone cannot distinguish related from unrelated trained models.

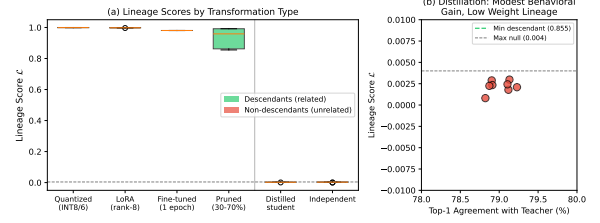


Figure 6: GPT-2-Small-Lite lineage benchmark (8 roots, 120 pairs across all splits; Table 7 reports results on the 3 held-out test roots, 45 pairs). (a) Lineage scores by transformation type. Descendants (green) score high; non-descendants (red) score near zero. (b) Distillation modestly improves teacher–student agreement (79% top-1, +1.4 points over independent training) while leaving the weight-lineage score near the independent-root range, confirming that behavioral similarity $\neq$ weight inheritance.

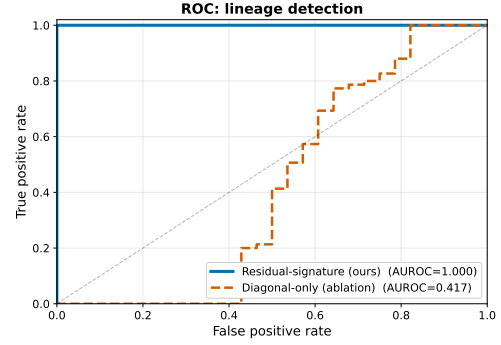


Figure 7: ROC for lineage verification on depth-24 residual MLPs. The residual-signature score achieves AUROC=1.000; trace concentration alone fails (AUROC=0.417) because it cannot distinguish two trained residual models.

D.6 Gradient-Based Suppression Attack

An adversary optimizes perturbation Δ to minimize lineage score subject to utility:

$$\min_{\Delta} \mathcal{L}_{\cos}(A, A + \Delta) + \lambda \cdot \|f_{A+\Delta}(X) - f_A(X)\|_2^2 \quad (11)$$

We run this attack on depth-24 MLPs ($d=64$). The null baseline for this configuration (score between the reference and an independent same-task model) is $\mathcal{L}_{\text{null}} = 0.084$.

E Transfer Beyond Language Models

The trace concentration phenomenon underlying our method is not specific to language models. Table 17 shows that architecture-aware branch products recover projection pairings in vision and speech architectures as well.

For ViT (Dosovitskiy et al., 2021), we use the

λ	Final $\mathcal{L}$	Eval loss	Verdict
0	0.053	39.5	utility destroyed
10^{-2}	0.054	0.77	+12% loss, below null
10^{-1}	0.083	0.70	+1.5% loss, $\approx$ null
≥ 1	0.91+	0.69	utility preserved

Table 16: Pareto frontier for suppression attack. The null baseline for this setup is $\mathcal{L}_{\text{null}} = 0.084$. Reaching the null threshold costs +1.5% utility loss ($\lambda=10^{-1}$); driving the score reliably below null ($\lambda=10^{-2}$) costs +12%.

Model	L	Pair Acc	AUC
ViT-B/16	12	100%	1.00
Whisper (tiny/base/sm)	4–12	100%	0.85
ResNet-50/101/152	5–35	91–100%	0.96

Table 17: Trace concentration in vision and speech architectures. Pair accuracy measures correct within-block projection recovery via Hungarian matching on $s(i, j)$. Random-init baselines: $\leq 9\%$.

same transformer MLP factorization as for language models (W_2W_1). For Whisper (Radford et al., 2023), we extract branch products from both encoder and decoder MLP blocks. For ResNet (He et al., 2016), bottleneck blocks require a three-layer product ($W_3W_2W_1$); naive two-layer factorization (W_3W_1 , skipping the middle conv) yields chance-level accuracy. On ImageNet-pretrained ResNet-50/101/152 (torchvision), the correct triple product recovers 91–100% of block pairings in layer3.

These results demonstrate that the underlying signal—trace concentration in residual branch products—is a general property of trained residual networks, not specific to language model architectures. This universality motivates building lineage signatures on the trace-concentrated structure. We focus the main evaluation on language models to match the ACL venue, but the method applies wherever residual blocks exist.